

Jarvis Core — Subsystem Specification

The memory, learning, and proactivity engine that makes Jarvis a personal assistant rather than a stateless chatbot. This document specifies **one subsystem** of the larger Jarvis architecture. It is the zoom-in behind the single "Jarvis Core" box in the system diagram.

Status: planning draft · feeds system-architecture design + Claude Code implementation.

1. What this is, and why it exists

Every AI assistant shares roughly the same LLM. The thing that differentiates Jarvis from a generic coding/chat agent is **a closed loop around one person that compounds over time**: it remembers, it builds a model of the user, and it acts on its own toward the user's goals. A general agent is reactive and has no durable model of *you*. Jarvis Core is the part that is *proactive* and *personal*.

Jarvis Core is built from two cooperating engines that share one memory:

- **Memory / learning engine** (the "forward" path): builds correct, structured knowledge about the user. Deterministic engines feed the LLM exact inputs so it reasons over facts, not guesses.
- **Evaluation engine** (the "proactive" path): generates things Jarvis *could* do, scores each by predicted usefulness, surfaces the best few, and tunes the scorer from feedback. This is a recommender/ranking system — the same machine that powers a content feed, pointed at the user's life instead of media.

The **evaluation/scoring function is the heart of the system**. Get it right and Jarvis is useful; get it wrong and it is noise.

2. Design foundations (prior art to study)

This subsystem deliberately reuses three battle-tested designs rather than inventing from scratch:

1. **Stanford "Generative Agents"** (arXiv:2304.03442) — the memory model: a memory stream of natural-language records, retrieval scored by **recency + importance + relevance**, and **reflection** that synthesizes raw observations into higher-level insights.
2. **Two-stage recommender** (Covington et al., YouTube, RecSys 2016) — the proactivity engine: **candidate generation** (high recall, cheap) followed by **ranking** (high precision, expensive), surfacing only the top few.
3. **Mulch** (memory layer for agents) — record design: **typed records, shelf-life tiers, confidence scoring, pruning of stale records, and pre-write redaction** of sensitive strings.

Supporting technique: **multi-armed bandits / Thompson sampling** for the explore-vs-exploit balance in the ranker.

3. Scope and boundaries

Jarvis Core IS:

- The memory stores (structured + vector) and their read/write logic.
- Reflection (pattern synthesis / learning).
- The user model (derived profile of interests, rhythms, goals, preferences).
- The proactivity engine (candidate generation → ranking → selection).
- The feedback loop.

Jarvis Core IS NOT (these are external components that *feed* the Core; they are specified elsewhere):

- **Orchestrator / Agent Core** — routing, tool dispatch, the reasoning loop.
- **Model Router** — local-vs-cloud decision + PII stripping.
- **Collectors / Connectors** — market, news, YC, calendar, finance fetchers.
- **Scheduler** — the cron that triggers briefings and proactive runs.
- **Voice pipeline, UI, clients.**

Interface contract with the outside:

- **Inputs:** `SignalEvent`s (from interactions + Collectors), and retrieval queries from the Orchestrator.
 - **Outputs:** retrieved memory context (to the Orchestrator), and ranked `Suggestion`s (to the delivery surface — feed/voice).
-

4. Deterministic vs LLM (the compute boundary)

Most of Jarvis Core is **deterministic code** — fast, free, testable, private. The LLM is called only where genuine language understanding/generation is required. This boundary is the whole point of the design.

Deterministic (plain code, no model call):

- Signal capture and storage.
- Retrieval scoring (recency decay, cosine relevance, weighting, top-K).

- Candidate generation (rules + queries over Collector data + user model).
- The ranking/evaluation function (feature math, thresholds, selection).
- Explore/exploit sampling.
- All feedback updates (importance bumps, confidence decay, weight tuning).

LLM-assisted (model call required):

- **Reflection** — synthesizing higher-level insights from raw memories (Brain).
- **Suggestion text** — writing the human-facing card/voice line (Brain, or small Heartbeat model for simple nudges).
- *(Optional)* **Importance scoring** at write time — can be a small classifier or a cheap LLM call; can also start as a heuristic.

If a task can be done with code, it is not given to the LLM.

5. Data model

Stack assumption: Python; SQLite for structured data; a vector DB (Chroma or Qdrant) for embeddings. Field types are generic.

5.1 `MemoryRecord` (the core object — episodic + semantic memory)

```
MemoryRecord {
  id:                uuid
  type:              enum[observation, preference, decision, pattern, outcome, reflection]
  content:           text                # natural-language description of the memory
  embedding:         vector              # for relevance / semantic retrieval
  created_at:        timestamp
  last_accessed_at: timestamp            # updated on each retrieval; drives recency
  importance:        float [0..1]       # "poignancy"; assigned at write time
  tier:              enum[foundational, tactical, observational] # shelf-life class
  confidence:        float [0..1]       # rises with re-confirmation, decays if
contradicted
  source:           enum[interaction, collector, reflection, feedback]
  links:            [uuid]              # related records (a reflection links to its
source observations)
  metadata:         json                # topic tags, project_id, entity refs, etc.
}
```

Tier semantics (shelf-life / pruning):

- `foundational` — durable facts about the user; effectively never expire.

- `tactical` — current context (this week's project, this month's goal); decays over weeks.
- `observational` — raw, low-level events; aggressively pruned once summarized by reflection.

5.2 Structured store (SQLite domain tables)

Hard, structured facts live in their own tables, **not** as `MemoryRecord`s: `calendar_events`, `transactions`, `tasks`, `goals`, `contacts`. The Core reads/writes these directly. (Distinction: episodic/semantic memory → vector store + `MemoryRecord`; exact structured facts → SQLite tables.)

5.3 `UserModel` (the derived profile — materialized by reflection)

```
UserModel {
  interests:    [{ topic, weight [0..1], confidence, last_updated }]
  rhythms:     [{ pattern, window, days, confidence }]    # e.g. deep_work 09:00-11:00
               Mon-Fri
  goals:       [{ id, description, status, progress, priority, deadline? }]
  preferences: [{ key, value, confidence }]                # e.g. suppress_topic: crypto
  dnd:         [{ window, days }]                          # do-not-disturb windows
  updated_at:  timestamp
}
```

A fast-read materialized view kept current by reflection (Stage 4). Backed by `MemoryRecord`s of type `pattern`/`preference`.

5.4 `SignalEvent` (Stage 1 output — the "engagement data")

```
SignalEvent {
  id:          uuid
  ts:          timestamp
  kind:        enum[query, suggestion_shown, suggestion_acted, suggestion_dismissed,
                  suggestion_ignored, item_dwell, collector_event, calendar_event, ...]
  payload:     json          # topic(s), project_id, item_ref, dwell_ms, etc.
  session_id:  uuid
}
```

5.5 `Suggestion` and `Outcome` (Stages 6–7)

```
Suggestion {
  id:          uuid
  created_at:  timestamp
  candidate_type: enum[market_move, free_time, followup_due, yc_launch, goal_nudge, ...]
  content:     text          # human-facing card/voice text (LLM-written)
  features:    json          # the feature vector used for scoring
}
```

```

score:          float          # ranking score
surfaced:       bool
channel:        enum[feed, voice, notification]
}

Outcome {
  suggestion_id: uuid
  ts:           timestamp
  result:       enum[acted, dismissed, ignored, more_like_this, less_like_this]
}

```

6. The pipeline (7 stages)

Data flows top to bottom; Stage 7 closes the loop back to Stages 2/5/6.

Stage 1 — Signal capture. Emits a structured `SignalEvent` for every interaction and Collector event. Dumb, cheap, always-on. *Must be live from Phase 2 so later stages have history to learn from.*

- In: interactions, Collector events. Out: `SignalEvent`s appended to an event log.

Stage 2 — Memory store. Persists memory. Significant signals become `MemoryRecord`s (with type, importance, tier, embedding); exact facts go to SQLite domain tables. Handles pruning of stale `observational` records.

- In: `SignalEvent`s, reflection outputs, feedback. Out: persisted memory.

Stage 3 — Retrieval. On demand, scores all candidate memories and returns the top-K that fit the context budget — never the whole store. Updates `last_accessed_at` on returned records.

- In: a query/situation from the Orchestrator. Out: top-K `MemoryRecord`s. Formula: §7.1.

Stage 4 — Reflection (the learning). Periodically (trigger in §7.4) reviews recent high-importance memories and asks the LLM to synthesize higher-level insights, which are written back as `reflection` records and used to update the `UserModel`. Raw observation is data; reflection is *knowing the user*.

- In: recent memories. Out: new `reflection` records + `UserModel` updates. Runs on the Brain (queued if asleep).

Stage 5 — User model. The standing, materialized profile (interests, rhythms, goals, preferences, DND) with per-item confidence that rises on re-confirmation and decays when contradicted.

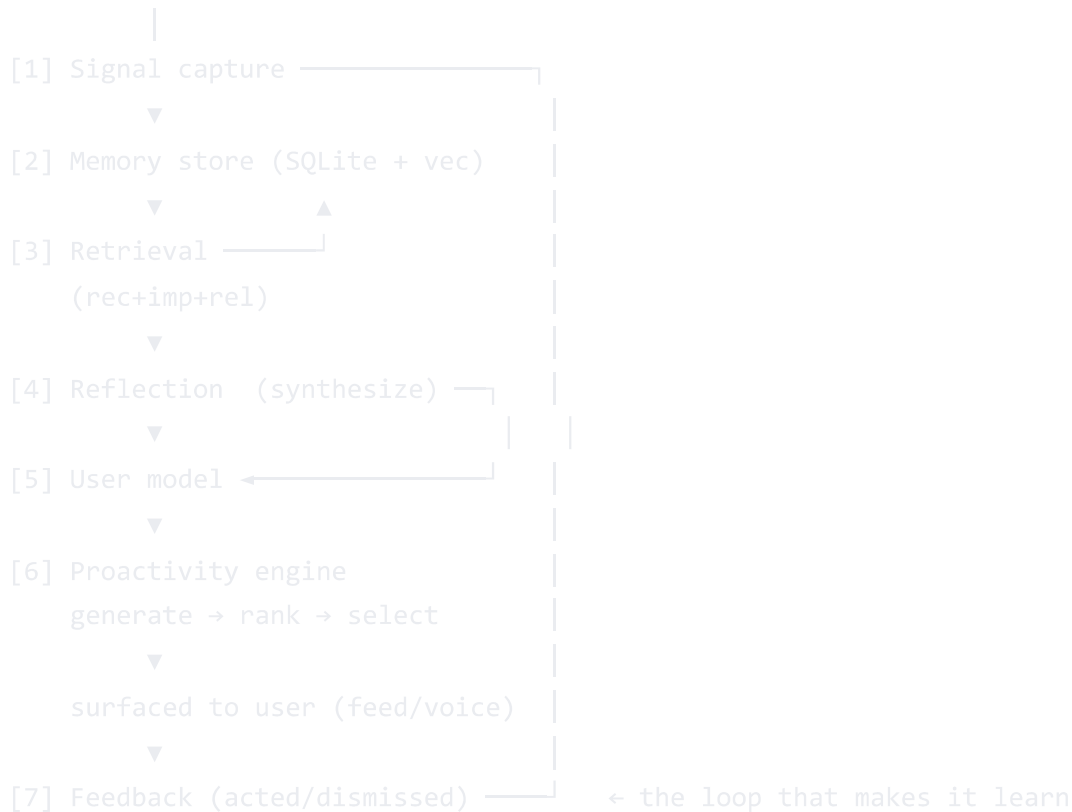
- In: reflection outputs, feedback. Out: a fast-read profile for ranking + briefings.

Stage 6 — Proactivity engine (two-stage recommender).

- *Candidate generation* (recall, cheap): given fresh Collector data + `UserModel`, produce a broad pool of things Jarvis could surface (watched stock moved, free hour before a meeting, promised follow-up overdue, relevant YC launch, goal nudge). Cast a wide net.
- *Ranking* (precision, the evaluation function): score each candidate by predicted usefulness *now* (§7.2), then surface only the top N (cap ~3 per window). Rank hard, show little.
- *Explore/exploit*: occasionally include an out-of-profile candidate to learn (§7.3).
- In: Collector data + `UserModel` + current context. Out: ranked `Suggestion`s to the delivery surface.

Stage 7 — Feedback loop. Every surfaced suggestion gets an `Outcome`, which updates memory importance (Stage 2), the user model (Stage 5), and the ranker's weights (Stage 6). This single loop is the difference between storing facts and getting smarter about the user. Mechanics in §7.5.

Collectors / interactions



7. Core formulas

7.1 Retrieval score (Stage 3) — from Generative Agents

$$\text{retrieval_score}(m, q) = w_{\text{rec}} \cdot \text{recency}(m) + w_{\text{imp}} \cdot \text{importance}(m) + w_{\text{rel}} \cdot \text{relevance}(m,$$

```

q)

recency(m)    = exp(-λ · hours_since(m.last_accessed_at))    # exponential decay; λ
tunable
importance(m) = m.importance                                # [0..1]
relevance(m)  = cosine_similarity(m.embedding, embed(q))

# normalize each term to [0..1] (min-max over the candidate set) before weighting
# default weights w_rec = w_imp = w_rel = 1.0 (tune later)
# return top-K records that fit the LLM context budget

```

7.2 Ranking / evaluation function (Stage 6) — the soul of the system

```

usefulness(c, ctx, user) =
    β_goal      · goal_relevance(c, user.goals)
  + β_urgency   · urgency(c)                    # time-sensitivity
  + β_interest  · interest_match(c, user.interests)
  + β_timing    · timing_fit(ctx.now, user.rhythms, user.dnd)
  + β_novelty   · exploration_bonus(c)           # explore term (see 7.3)
  - β_fatigue   · recent_interruption_penalty(ctx) # suppress spam

# surface candidates where usefulness > threshold; cap N per window (e.g. 3)
# β weights: start hand-set; LEARN them from feedback (7.5) once data exists.
# Begin with a linear model; graduate to logistic regression or gradient-boosted
# trees (e.g. LightGBM) as the labeled outcome set grows.

```

Hard constraint on the learning target — see §8.

7.3 Explore vs exploit (Stage 6)

With small probability ϵ (or via Thompson sampling), include one candidate outside the user's confirmed interests and log its **Outcome**. Without this, the model filter-bubbles and never discovers a new interest. The exploration rate should decay as confidence in the user model rises.

7.4 Reflection trigger (Stage 4)

```

when sum(importance of new memories since last reflection) >= REFLECTION_THRESHOLD:
    pull recent high-importance memories
    LLM(Brain): "What higher-level patterns or insights follow from these observations?"
    write each insight as MemoryRecord(type=reflection, tier=tactical|foundational,
links=[sources])
    update UserModel (interests / rhythms / goals / preferences)

```

7.5 Feedback updates (Stage 7)

- `(acted)` / `(more_like_this)` → raise importance of linked memories; increase the topic's interest weight; **positive** training label for the ranker.
 - `(dismissed)` / `(less_like_this)` → decay importance; lower the interest weight; **negative** label; optionally add a `(suppress_topic)` preference.
 - `(ignored)` → **weak** negative — likely a timing/fatigue miss rather than irrelevance; adjust `(timing_fit)`/`(fatigue)` more than relevance.
-

8. The objective constraint (non-negotiable)

A content feed optimizes its evaluation function for **engagement** (time-on-app), which is exactly why feeds are designed to be addictive. Jarvis uses the same *machinery* with a different *objective*:

The ranker's learning target is USEFULNESS — did this save time, advance a goal, or prevent a miss — NOT engagement or time-on-app.

Same architecture, healthier reward signal. This is what keeps Jarvis a tool instead of another thing competing for the user's attention. Any metric, label, or reward used to tune §7.2 must encode usefulness, never attention capture.

9. Deployment (which machine runs what)

- **Heartbeat (HP Pavilion, always-on 24/7):** Signal capture, memory store, retrieval, the ranking/evaluation function, explore/exploit, feedback ingestion, and a small CPU model for simple nudge text. So Jarvis can watch and decide whether to interrupt around the clock without waking the GPU rig.
 - **Brain (RTX 5080, on-demand):** Reflection (LLM synthesis) and richer suggestion composition. These are queued via the Task/Message Queue and run when the Brain is awake. Reflection is a natural "run when the Brain wakes" job.
 - **Embedding** of new memories: a small embedder on the Heartbeat, or queued for the Brain.
-

10. Build phasing (within the Core)

- **Phase 2:** Signal capture + memory store (read/write) + basic retrieval. *Turn signal capture on now — the Phase 5 learning is worthless without accumulated history.*
- **Phase 3:** Delivery surface (the "Jarvis feed" card UI + voice channel) so suggestions have somewhere to go.

- **Phase 5:** Reflection + user model + proactivity engine (candidate generation + ranking) + feedback loop + explore/exploit.

Sequencing rule: **signal capture (Phase 2) must precede the learning logic (Phase 5) that consumes it.**

11. Open decisions (defer to implementation, but track)

- Recency decay constant λ ; reflection importance threshold.
 - Ranker model: start hand-weighted linear $\rightarrow$ graduate to logistic regression / GBDT once labeled outcomes exist.
 - Embedding model choice (local).
 - Whether `UserModel` is fully materialized or partly computed on read.
 - Importance scoring at write time: heuristic vs small classifier vs cheap LLM call.
 - Exploration mechanism: fixed ϵ vs Thompson sampling; decay schedule.
-

12. References to study before building

- Park et al., **Generative Agents: Interactive Simulacra of Human Behavior**, arXiv:2304.03442 — memory stream, recency/importance/relevance retrieval, reflection.
- Covington et al., **Deep Neural Networks for YouTube Recommendations**, RecSys 2016 — two-stage candidate generation + ranking; two-tower retrieval.
- **Mulch** (github.com/jayminwest/mulch) — typed records, tiers, confidence, pruning, pre-write redaction.
- Multi-armed bandits / **Thompson sampling** — explore vs exploit.